

Python Programming

Unit 02 – Lecture 04 Notes

Functions Basics and Parameters

Tofik Ali

February 17, 2026

Contents

1	Lecture Overview	2
2	Why Functions?	2
3	Function Syntax and Behavior	2
3.1	Basic Definition	2
3.2	<code>return</code> vs <code>print</code>	3
3.3	Docstrings	3
4	Parameters and Arguments	3
4.1	Parameters vs Arguments	3
4.2	Positional and Keyword Arguments	3
5	Default Arguments	3
5.1	Pitfall: Mutable Default Arguments	4
5.2	Safe Pattern for Defaults	4
6	Variable-Length Arguments	4
6.1	<code>*args</code> (Extra Positional Arguments)	4
6.2	<code>**kwargs</code> (Extra Keyword Arguments)	4
6.3	Worked Example: Average of Any Number of Scores	4
7	Scope: Local vs Global	5
8	Demo Walkthrough: Grade Calculator Function	5
8.1	What the Demo Is Teaching	5
8.2	Suggested Run Steps	5
9	Quiz and Solutions	5
9.1	Checkpoint 1 (Mutable Defaults)	5
9.2	Checkpoint 2 (Local Scope)	5
9.3	Think-Pair-Share (Keyword Arguments)	6
9.4	Exit Question (Signature for Average)	6

10 Practice Exercises (With Solutions)	6
10.1 Exercise 1: Safe Append Helper	6
10.2 Exercise 2: Named Formatting Function	6
10.3 Exercise 3: Return vs Print	6
11 Further Reading	7

1 Lecture Overview

Functions let you name a block of logic and reuse it. This is one of the most important habits for writing clean programs: instead of copying code, you call a function.

This lecture covers:

- function definition syntax (`def`, parameters, `return`),
- docstrings and why documentation matters,
- positional vs keyword arguments,
- default arguments (and the common mutable-default trap),
- variable-length arguments (`*args`, `**kwargs`), and
- local vs global scope.

2 Why Functions?

Functions improve code quality in multiple ways:

- **Avoid repetition:** write logic once, reuse many times.
- **Readability:** a good name summarizes the intent.
- **Testing:** small functions are easier to test than long scripts.
- **Abstraction:** callers do not need to know internal details.

3 Function Syntax and Behavior

3.1 Basic Definition

```
def add(a, b):
    return a + b

result = add(3, 4)
```

Key parts:

- `def` starts a function definition.
- Parameters (`a`, `b`) are inputs.
- `return` sends a value back to the caller.

3.2 return vs print

A common beginner confusion is mixing output with return values.

- **return** gives a value back to the caller for further use.
- **print** only displays text; it does not produce a reusable value.

3.3 Docstrings

A docstring is the first string in a function body. It documents the purpose, inputs, and output.

```
def area(r):  
    """Return the area of a circle with radius r."""  
    return 3.14 * r * r
```

Docstrings can be viewed with `help(area)` or via `area.__doc__`.

4 Parameters and Arguments

4.1 Parameters vs Arguments

- **Parameters** appear in the function definition.
- **Arguments** are the actual values passed during a call.

Example:

```
def greet(name, msg): # parameters  
    return f"{msg}, {name}"  
  
print(greet("Asha", "Hello")) # arguments
```

4.2 Positional and Keyword Arguments

```
def power(base, exp):  
    return base ** exp  
  
power(2, 3) # positional  
power(base=2, exp=3) # keyword  
power(exp=3, base=2) # keyword order can change
```

Keyword arguments improve readability when a function has many parameters.

5 Default Arguments

Defaults make a function flexible:

```
def greet(name, msg="Hello"):  
    return f"{msg}, {name}"
```

Important detail: default values are evaluated once, when the function is defined (not every time it is called).

5.1 Pitfall: Mutable Default Arguments

This is one of the most common Python interview and exam traps.

```
def add_item(x, items=[]):
    items.append(x)
    return items

print(add_item(1)) # [1]
print(add_item(2)) # [1, 2] (surprise!)
```

Why it happens: the default list `[]` is created once and reused.

5.2 Safe Pattern for Defaults

Use `None` and create a new list inside the function:

```
def add_item_safe(x, items=None):
    if items is None:
        items = []
    items.append(x)
    return items
```

6 Variable-Length Arguments

Sometimes you do not know how many inputs the caller will provide.

6.1 *args (Extra Positional Arguments)

```
def total(*nums):
    return sum(nums)

print(total(10, 20, 30))
```

Inside the function, `nums` is a tuple of all extra positional arguments.

6.2 **kwargs (Extra Keyword Arguments)

```
def describe(**info):
    return info

print(describe(name="Asha", age=18))
```

Inside the function, `info` is a dictionary of keyword arguments.

6.3 Worked Example: Average of Any Number of Scores

```
def avg(*scores):
    if not scores:
        return 0.0
    return sum(scores) / len(scores)
```

The empty-input check avoids division by zero.

7 Scope: Local vs Global

By default, variables created inside a function are **local**.

```
x = 5

def foo():
    x = 2 # local
    return x

print(foo(), x)
```

Using global variables is possible (with the `global` keyword), but it makes code harder to reason about. Prefer returning values instead.

8 Demo Walkthrough: Grade Calculator Function

Script: `demo/grade_calculator.py`

8.1 What the Demo Is Teaching

- Use a function to keep grading logic in one place.
- Use parameters/defaults so the grading scheme can be reused.
- Return a grade value (instead of only printing).

8.2 Suggested Run Steps

```
python demo/grade_calculator.py
```

Extension idea: allow custom grade boundaries (e.g., a different university policy).

9 Quiz and Solutions

9.1 Checkpoint 1 (Mutable Defaults)

Question. What happens if you use a mutable default like `def f(items=[])`?

Solution. The default object is created once and reused across calls. So changes to `items` in one call affect the default in later calls.

9.2 Checkpoint 2 (Local Scope)

Question. Predict the output:

```
x = 5

def foo():
    x = 2
    return x

print(foo(), x)
```

Solution. Output is 2 5. The `x` inside `foo` shadows the global `x` but does not change it.

9.3 Think-Pair-Share (Keyword Arguments)

Prompt. When do keyword arguments improve readability?

Sample response. When a function has multiple parameters of the same type or meaning, keywords prevent confusion. For example, `resize(width=640, height=480)` is clearer than `resize(640, 480)`.

9.4 Exit Question (Signature for Average)

Question. Write a function signature that accepts any number of scores and returns their average.

Solution.

```
def avg(*scores):  
    if not scores:  
        return 0.0  
    return sum(scores) / len(scores)
```

10 Practice Exercises (With Solutions)

10.1 Exercise 1: Safe Append Helper

Task. Write a function `push(x, items=None)` that appends `x` to a list, but does not use a mutable default.

Solution.

```
def push(x, items=None):  
    if items is None:  
        items = []  
    items.append(x)  
    return items
```

10.2 Exercise 2: Named Formatting Function

Task. Write `format_name(first, last, title="")` that returns a formatted full name. Prefer keyword arguments for optional parts.

Solution.

```
def format_name(first, last, title=""):  
    title = title.strip()  
    if title:  
        return f"{title} {first} {last}"  
    return f"{first} {last}"
```

10.3 Exercise 3: Return vs Print

Task. Write a function `is_even(n)` that returns `True` or `False`. Do not print inside the function.

Solution.

```
def is_even(n):  
    return n % 2 == 0
```

11 Further Reading

- <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

Appendix: Slide Deck Content (Reference)

The material below is a reference copy of the slide deck content. Exercise solutions are explained in the main notes where applicable.

Title Slide

Repository: <https://github.com/tali7c/Python-Programming>

Quick Links

[Core Concepts](#) [Demo](#) [Interactive](#) [Summary](#)

Agenda

- Overview
- Core Concepts
- Demo
- Interactive
- Summary

Learning Outcomes

- Define reusable functions with parameters and return values
- Use keyword, default, and variable-length arguments
- Write docstrings to document behavior
- Explain local vs global scope

Why Functions?

- Avoid repetition
- Improve readability
- Make code easier to test
- Enable modular design

Function Syntax

```
def add(a, b):  
    return a + b  
  
result = add(3, 4)
```

- `def` creates a function
- `return` sends a value back

Docstrings

- Triple-quoted string inside a function
- Describe purpose, parameters, and return value

```
def area(r):  
    """Return area of a circle."""  
    return 3.14 * r * r
```

Parameters vs Arguments

- Parameters appear in function definition
- Arguments are values passed in a call
- Positional and keyword arguments both valid

Default Arguments

- Provide a fallback value
- Evaluated at function definition time
- Avoid mutable defaults (like lists)

```
def greet(name, msg="Hello"):  
    return f"{msg}, {name}"
```

Pitfall: Mutable Default Arguments

- Default objects are created once and reused
- A list default is shared across calls

```
def add_item(x, items=[]):  
    items.append(x)  
    return items  
  
print(add_item(1)) # [1]  
print(add_item(2)) # [1, 2] (surprise!)
```

Variable-Length Arguments

- `*args` collects extra positional arguments
- `**kwargs` collects extra keyword arguments

```
def total(*nums):  
    return sum(nums)
```

Worked Example: Average of Any Number of Scores

```
def avg(*scores):
    if not scores:
        return 0.0
    return sum(scores) / len(scores)

print(avg(10, 20, 30))
```

- Handle empty input to avoid division by zero

Scope: Local vs Global

- Variables inside functions are local by default
- Use `global` only when necessary
- Prefer returning values instead of modifying globals

Demo: Grade Calculator Function

- Compute average score
- Return letter grade based on thresholds
- Use default parameters for flexibility
- Extension: allow custom boundaries for other grading schemes

Script: `demo/grade_calculator.py`

Checkpoint 1

What happens if you use a mutable default like `def f(items=[])`?

Checkpoint 2

Predict the output:

```
x = 5

def foo():
    x = 2
    return x

print(foo(), x)
```

Think-Pair-Share

When do keyword arguments improve readability? Give a short example.

Summary

- Functions organize and reuse logic
- Arguments can be positional, keyword, or variable-length
- Docstrings document intent and usage
- Scope rules help avoid bugs

Exit Question

Write a function signature that accepts any number of scores and returns their average.